

esbuild internals learning roadmap

4-week deep dive into bundler internals via esbuild

prerequisites check

- ☐ node.js installed
 - ☐ go installed (version 1.20+)
 - ☐ git, code editor
 - ☐ ability to read go code (you'll learn as you go)
-

week 1: bundler fundamentals + esbuild setup

day 1: understand the problem

what bundlers solve:

- module systems (ESM, CommonJS, AMD)
- dependency resolution
- code transformation (TS → JS, JSX → JS)
- tree shaking
- code splitting
- minification

read these (3 hours total):

1. <https://nodejs.org/api/esm.html> - module resolution algorithm
2. <https://webpack.js.org/concepts/module-resolution/> - how webpack resolves
3. <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Modules> - ESM basics

task: write a 200-word explanation of "why bundlers exist"

day 2: module resolution by hand

goal: understand how import statements become file paths

exercise (2 hours): create this file structure:

```
project/  
  src/  
    index.js (imports './utils')  
    utils.js (imports 'lodash')  
  node_modules/  
    lodash/  
      package.json (main: "index.js")  
      index.js
```

manually trace:

1. how `import { add } from './utils'` resolves to `src/utils.js`
2. how `import _ from 'lodash'` resolves to `node_modules/lodash/index.js`
3. what happens with `.js`, `.ts`, `.jsx` extensions

write: document the resolution algorithm in your own words

day 3: AST basics

goal: understand how code becomes data structures

read (2 hours):

- <https://astexplorer.net/> - visualize ASTs
- <https://github.com/jamiebuilds/the-super-tiny-compiler> - read entire repo

exercise:

```
javascript
```

```
// install: npm install @babel/parser @babel/traverse
const parser = require('@babel/parser');
const traverse = require('@babel/traverse').default;

const code = `
import { useState } from 'react';
const name = 'Chetan';
`;

const ast = parser.parse(code, { sourceType: 'module' });

// task: extract all import statements
traverse(ast, {
  ImportDeclaration(path) {
    console.log(path.node.source.value); // should print 'react'
  }
});
```

deliverable: script that extracts all imports from a JS file

day 4: set up esbuild

clone and explore:

```
bash

git clone https://github.com/evanw/esbuild.git
cd esbuild
```

read these files (just read, don't understand everything):

- `README.md`
- `docs/architecture.md` (CRITICAL - read twice)
- `internal/bundler/bundler.go` (skim structure)

task: map esbuild's architecture to these concepts:

- scanning (finding files)
- parsing (code → AST)
- linking (connecting dependencies)
- bundling (combining files)

write: 300-word summary of esbuild's architecture

day 5: esbuild's module resolution

goal: understand how esbuild resolves imports

read source:

- `internal/resolver/resolver.go` (main resolution logic)
- focus on `func (r *Resolver) Resolve()` method

compare with:

- node's resolution algorithm (from day 1)
- note differences

exercise: create test cases:

```
javascript

// test1.js
import './file.js' // explicit extension

// test2.js
import './file'    // implicit extension

// test3.js
import 'react'     // node_modules
```

run: `esbuild test1.js --bundle --outfile=out1.js`

task: trace how esbuild resolves each import by reading resolver code

day 6-7: build mini resolver

project: implement basic import resolver in node.js

requirements:

javascript

// your resolver should handle:

`resolve('./utils.js', '/project/src/index.js')`

// → /project/src/utils.js

`resolve('lodash', '/project/src/index.js')`

// → /project/node_modules/lodash/index.js

`resolve('./utils', '/project/src/index.js')`

// → /project/src/utils.js (try .js, .ts, .jsx)

implementation hints:

- use `path.resolve()` for relative paths
- use `path.dirname()` to get directory
- check file extensions in order: `.js`, `.ts`, `.jsx`, `.tsx`
- search `node_modules` recursively upward

deliverable:

- working resolver (100-150 lines)
- test cases proving it works
- blog post: "how module resolution works"

week 2: parsing and transformation

day 8: understand go basics for esbuild

learn just enough go to read esbuild:

read (3 hours):

- "a tour of go" - <https://go.dev/tour/>
 - skip: methods, interfaces, concurrency (for now)
 - focus: syntax, structs, functions, slices, maps

specific to esbuild:

- how structs work (esbuild uses heavily)
- pointer syntax (`(*)` and `(&)`)
- method receivers
- error handling patterns

don't build anything in go yet. just learn to read it.

day 9-10: esbuild's parser

goal: understand how esbuild parses javascript

read source:

- `internal/js_parser/js_parser.go`
- `internal/js_lexer/js_lexer.go` (tokenization)

key functions:

- `Parse()` - main entry point
- `parseStmt()` - statement parsing
- `parseExpr()` - expression parsing

trace this code manually:

```
javascript

import { useState } from 'react';
const name = 'Chetan';
```

deliverable: document the parsing flow:

1. code $\rightarrow$ tokens (lexer)
2. tokens $\rightarrow$ AST (parser)
3. what AST nodes are created

day 11-12: transformation pipeline

goal: understand how esbuild transforms code

read:

- `internal/js_parser/js_parser.go` - look for transformation logic
- search for: "JSX", "TypeScript", "visitStmts"

key transformations:

- JSX → `React.createElement()`
- TS types → removed
- ESM → CommonJS (when targeting node)

trace:

```
jsx

// input
const Button = () => <button>Click</button>;

// esbuild transform
// → what does output look like?
```

exercise: run esbuild with different targets:

```
bash

esbuild input.jsx --loader=jsx --target=es2020
esbuild input.jsx --loader=jsx --target=es5
esbuild input.ts --loader=ts
```

compare outputs. understand why they differ.

deliverable: document 3 major transformations esbuild performs

day 13-14: build mini parser

project: parse JS and extract imports + exports

use: `@babel/parser` (don't write a parser from scratch)

requirements:

javascript

// your parser should extract:

```
const result = parse(`
  import React from 'react';
  import { useState } from 'react';
  export const name = 'Chetan';
  export default App;
`);

// result:
// {
//   imports: [
//     { source: 'react', specifiers: ['default as React'] },
//     { source: 'react', specifiers: ['useState'] }
//   ],
//   exports: [
//     { name: 'name' },
//     { name: 'default' }
//   ]
// }
```

deliverable:

- working parser
- handles: named imports, default imports, namespace imports
- handles: named exports, default exports
- blog post: "how JavaScript modules work"

week 3: dependency graph and bundling

day 15-16: dependency graphs

goal: understand how esbuild builds dependency graphs

read:

- [internal/graph/graph.go](#)
- [internal/bundler/linker.go](#)

key concepts:

- entry points
- module graph (nodes = files, edges = imports)
- circular dependencies
- code splitting points

visualize: create this structure:

```
index.js → utils.js → helpers.js
      → components.js → utils.js (circular!)
```

exercise:

```
bash
esbuild index.js --bundle --metafile=meta.json
```

read `meta.json` - it contains the dependency graph.

task: write code to visualize the graph (use any format - text tree, mermaid, whatever)

day 17-18: code splitting

goal: understand how esbuild splits code

read:

- `internal/bundler/bundler.go` - search for "splitting"
- `internal/bundler/linker.go` - look for chunk generation

key concepts:

- entry point chunks
- shared chunks
- dynamic imports

exercise:

```
javascript
// index.js
import('./lazy.js').then(mod => console.log(mod));

// run:
esbuild index.js --bundle --splitting --format=esm --outdir=dist
```

analyze:

- how many files generated?
- what's in each file?
- how does esbuild decide what to split?

deliverable: explain esbuild's code splitting algorithm

day 19-20: tree shaking

goal: understand dead code elimination

read:

- [internal/bundler/linker.go](https://internal.bundler.linker.go) - search for "tree shaking"
- look for symbol tracking logic

trace:

```
javascript

// utils.js
export const used = () => 'used';
export const unused = () => 'unused';

// index.js
import { used } from './utils.js';
console.log(used());

// after bundle: unused() should be removed
```

exercise: test tree shaking:

```
bash

esbuild index.js --bundle --minify --outfile=out.js
```

check: is `unused` in `out.js`?

why it works:

- esbuild tracks which exports are imported
- removes unused exports
- requires ESM (not CommonJS)

deliverable: explain when tree shaking works and when it doesn't

day 21: build mini bundler

project: combine resolver + parser into basic bundler

requirements:

```
javascript
```

```
// input:
```

```
// index.js: import { add } from './math.js';
```

```
// math.js: export const add = (a, b) => a + b;
```

```
// output: single file with both modules
```

algorithm:

1. parse entry file (extract imports)
2. resolve each import to file path
3. parse each imported file
4. build dependency graph
5. concatenate files in correct order
6. wrap in module system (IIFE or similar)

deliverable:

- working bundler (200-300 lines)
- handles: multiple files, nested imports
- doesn't need: code splitting, tree shaking (yet)
- blog post: "i built a JavaScript bundler"

week 4: optimization and performance

day 22-23: parallelization in esbuild

goal: understand why esbuild is fast

read:

- `internal/bundler/bundler.go` - search for goroutines
- esbuild blog: <https://esbuild.github.io/faq/#benchmark-details>

key techniques:

- go's concurrency (goroutines)
- parallel parsing
- parallel transformation
- shared memory (no serialization)

compare: webpack (JavaScript, single-threaded) vs esbuild (Go, multi-threaded)

task: benchmark parsing time

```
bash

# create 100 JS files
# measure time to parse all with node vs esbuild
```

deliverable: document 3 ways esbuild achieves speed

day 24-25: minification

goal: understand code compression

read:

- `internal/js_printer/js_printer.go` - code generation
- `internal/renamer/renamer.go` - symbol mangling

techniques:

- whitespace removal
- identifier shortening (`myLongVariableName` → `a`)
- dead code elimination
- constant folding

exercise:

```
bash

esbuild input.js --minify --outfile=min.js
esbuild input.js --outfile=normal.js
```

compare outputs. identify each optimization.

deliverable: explain 5 minification techniques

day 26-27: source maps

goal: understand debugging bundled code

read:

- `internal/sourcemap/sourcemap.go`
- source map spec: <https://sourcemap.info/spec.html>

how they work:

- VLQ encoding
- mappings array
- original position → generated position

exercise:

```
bash

esbuild input.js --bundle --sourcemap --outfile=out.js
# generates out.js.map
```

task:

- open browser devtools
- set breakpoint in original file
- see it map to bundled code

deliverable: explain source map format

day 28: capstone project

build: feature-complete bundler

requirements:

-  module resolution (relative, node_modules)
-  parsing (extract imports/exports)
-  dependency graph
-  bundling (single output file)
-  basic minification (remove whitespace)
-  handle circular dependencies
-  error messages (file not found, syntax errors)

optional:

- JSX transformation
- TypeScript stripping
- source maps
- watch mode

deliverable:

- github repo
 - README with examples
 - blog post: "what i learned building a bundler"
-

resources summary

must-read

- esbuild source: <https://github.com/evanw/esbuild>
- esbuild architecture: <https://github.com/evanw/esbuild/blob/main/docs/architecture.md>
- module resolution: <https://nodejs.org/api/esm.html>
- AST explorer: <https://astexplorer.net>

go language

- tour of go: <https://go.dev/tour/>
- effective go: https://go.dev/doc/effective_go
- don't read go books - learn from esbuild source

bundler concepts

- webpack concepts: <https://webpack.js.org/concepts/> (theory)
- rollup guide: <https://rollupjs.org/guide/en/> (tree shaking explained well)

avoid

- youtube tutorials (too surface-level)
 - medium articles (usually shallow)
 - courses (you need source code, not lectures)
-

tracking progress

daily checklist

- ☐ 1 hour reading (docs/source)
- ☐ 30 min implementing
- ☐ 30 min writing (notes/blog)

weekly deliverables

- week 1: module resolver + blog post
- week 2: parser + blog post
- week 3: mini bundler + blog post
- week 4: full bundler + summary post

success metrics

- can explain how bundlers work to another developer
 - can read esbuild source and understand most of it
 - built a working bundler (even if basic)
 - comfortable reading go code
 - 4 technical blog posts published
-

what comes after

immediate next steps

- read webpack source (compare with esbuild)
- learn go properly (build side projects)
- study V8 internals (week 5-8 from previous plan)

career leverage

you'll be able to:

- debug bundler issues at work
 - optimize build pipelines
 - contribute to build tools
 - understand framework internals (Next.js, Vite)
 - make architectural decisions about tooling
-

common mistakes to avoid

1. **trying to understand everything at once**

- read code, accept 40% understanding, move on
- understanding comes with repetition

2. **not building**

- reading without implementing = shallow learning
- every 2 days of reading = 1 day of building

3. **skipping documentation**

- write notes as you learn
- if you can't explain it, you don't understand it

4. **perfectionism**

- your bundler will be basic - that's fine
- goal is understanding, not production tool

5. **comparing with tutorials**

- tutorials skip the hard parts
 - source code is messy - that's real engineering
-

motivation reminders

why this matters

- bundlers are fundamental to modern web development
- deep understanding separates you from AI-assisted developers
- skills transfer: parsers, compilers, build tools all use similar concepts

when stuck

- being confused = learning
- skip confusing parts, return later
- every engineer finds source code hard initially

accountability

- commit to 1 hour daily for 28 days
- tell someone about your progress weekly
- publish blog posts (forces clarity)

start tomorrow. read esbuild architecture docs. clone the repo. don't overthink it.